

PROJECT INTRODUCTION -

This project focuses on analysing the e-commerce data.

In this project, we prepare a report to analyse the order and delivery performance by combining the frontend and the backend metrics. The objective is to track the business performance such as order count, customer count, revenue generated, delivery efficiency for different time periods (day, month, year).

The analysis provides insight into the business by comparing the delivery and estimated date and calculating the delays and measuring their impact on revenues. It also gives revenue generated from the delayed order.

Additionally, this project highlights regional performance by analysing revenue generated at state levels. It identifies high-performing products and evaluates regions contributing the maximum and minimum revenue. Furthermore, statistical measures such as mean, median, and mode are calculated to provide deeper insights into revenue distribution.

Overall, this dashboard enables data-driven decision-making to optimise delivery timelines, improve customer satisfaction, and maximise revenue.

DATA EXPLORATION

The dataset was collected from Kaggle and represents an e-commerce business environment. It includes detailed information on orders, customers, and revenue transactions. This data is used to analyse business performance, customer behaviour, and overall sales trends.

OPERATION -

-SQL DATA PROCESSING FLOW -

The SQL data processing phase focuses on cleaning, transforming, and preparing the dataset for analysis and visualization. The following steps were performed:

1. **Data Cleaning:** Identified and handled missing (NULL) values to ensure data consistency and accuracy.
2. **Statistical Analysis:** Calculated the **mean**, **median**, and **mode** to understand the distribution and central tendency of the data.
3. **Front End Analysis :**
 - **Total Orders:** Count of all orders placed.
 - **Total Customer Visits:** Count of unique customers who placed orders.
 - **Total Revenue:** Overall revenue generated from completed orders.

- **Avg_Revenue_Order** : Revenue generated per order.
- **Avg_revenue_customer** : Revenue generated per customer.
- 4. **Backend Analysis** :
 - Identified whether each delivery was **On Time** or **Delayed**.
 - Calculated the **number of delay days** for late deliveries.
 - Measured delivery performance using on-time and delayed delivery counts.
 - Analyzed the **revenue generated from on-time and delayed orders** to evaluate the business impact of delivery performance.
- 5. **Regional performance** : Regional analysis was performed to identify sales and delivery trends across different locations:
 - Determine the **region with the highest sales**.
 - Identified the **product generating the maximum and minimum revenue**.
 - Analyzed the **state-wise distribution of revenue**.
 - Identified the **region with the highest number of delayed deliveries**.
 - Determine the **region with the highest number of on-time deliveries** to evaluate operational efficiency.
- 1. **Data Cleaning** : we observe that at some deliveries actual deliveries date is not given so first we find the count where deliveries are NULL.

```
select count(*) as total_count , count(case when
    order_delivered_customer_date is not null
    and trim(order_delivered_customer_date) <> "" then 1
    end) as delivery_count ,
    count(case when order_estimated_delivery_date is not null
    And trim(order_estimated_delivery_date) <> "" then 1
    end) as estimated_count
from orders_dataset
```

Output:

- **Total Count:** 99,441
This is total no's. of records
- **Delivery Count:** 96,476
This is no's of records that were successfully delivered.
- **Estimated Count:** 99,44
This is estimated no's of records for which we have to committed to customer for the order delivery.

Since the **Delivery Count** (96,476) is lower than the **Total Count** (99,441), it indicates that some records have **NULL** or empty (") values in the **Delivery Date** field.

Because of these missing values, we cannot derive accurate insights or evaluate the backend metrics correctly. To resolve this, we should calculate the **latest Estimated Delivery Date** for records where the Delivery Date is null or empty and use it as a replacement. After filling these missing values, we can re-evaluate the backend metrics to obtain more accurate and meaningful insights.

```
latest_date as
(
  select max(nullif(trim(order_estimated_delivery_date), '')::date) as
    latest_delivery
  from orders_dataset),
```

Before calculating the backend metrics, the Latest Delivery Date logic should be applied to retrieve the most recent non-NULL Actual Delivery Date and the most recent NON-NULL Estimated Delivery Date for each record. This ensures that missing delivery dates are populated with the latest available values, allowing the metrics to be evaluated accurately and producing the correct output.

```
delivery_report as
(
  select od.order_id,nullif(trim(od.order_estimated_delivery_date),
    ''):: date as estimated_date,
    coalesce(nullif(trim(od.order_delivered_customer_date),
    '')::date,ld.latest_delivery::date) as final_delivery_date
  from orders_dataset od
  cross join latest_date ld
),
```

Final delivery reports

```
final_delivery_report as
(
  select count(*) , count(actual_estimated_date) as estimated_date ,
    count(final_delivery_date) as final_delivery_date
  from delivery_report),
-- count = 99441 ,count(estimated date) = 99441 , count(final_delivery_date) = 99441
```

- **Total Count:** 99,441
This is total no's. of records
- **Delivery Count:** 99441
This is no's of records that were successfully delivered.
- **Estimated Count:** 99,441
This is estimated no's of records for which we have to committed to customer for the order delivery.

2. Statistical Analysis :

- **Front-end :** For Revenue

CODE -

```
select min(payment) as min_revenue_q0 ,  
       percentile_cont(0.25) within group(order by  
payment) as q1,  
       percentile_cont(0.5) within group(order by  
payment) as median_q2,  
       percentile_cont(0.75)within group(order by  
payment) as q3,  
       max(payment) as max_revenue_q4  
from (select distinct coalesce(order_id, 'a') as order_id , sum(payment_value) as  
       payment  
       from orderpayment  
       group by 1  
       order by 2)
```

- Output :

Min_revenue_q0 Minimum Revenue	0
Q1 25th Percentile 1st Quartile	62.0099983215332
median_Q2 Median Revenue	105.29000091552734
Q3 75th Percentile 3rd Quartile	176.97000122070312
max_revenue_Q4 Maximum Revenue	13664.08

3. Front End Analysis-

In this section, we calculate key metrics and build business performance insights across different time periods, including weekly , monthly, and yearly analysis.

-Group data dynamically by week, Month, or Year using Excel Slicers or Formulas

-METRICS

A)Total Orders (cnt_order)

B)Total Customers (cnt_customer)

C)Total Revenue (revenue)

D)Avg Revenue per Order

E)Avg Revenue per Customer

CODE -

```
select date_trunc('month',od.order_purchase_timestamp)::date
as order_period,

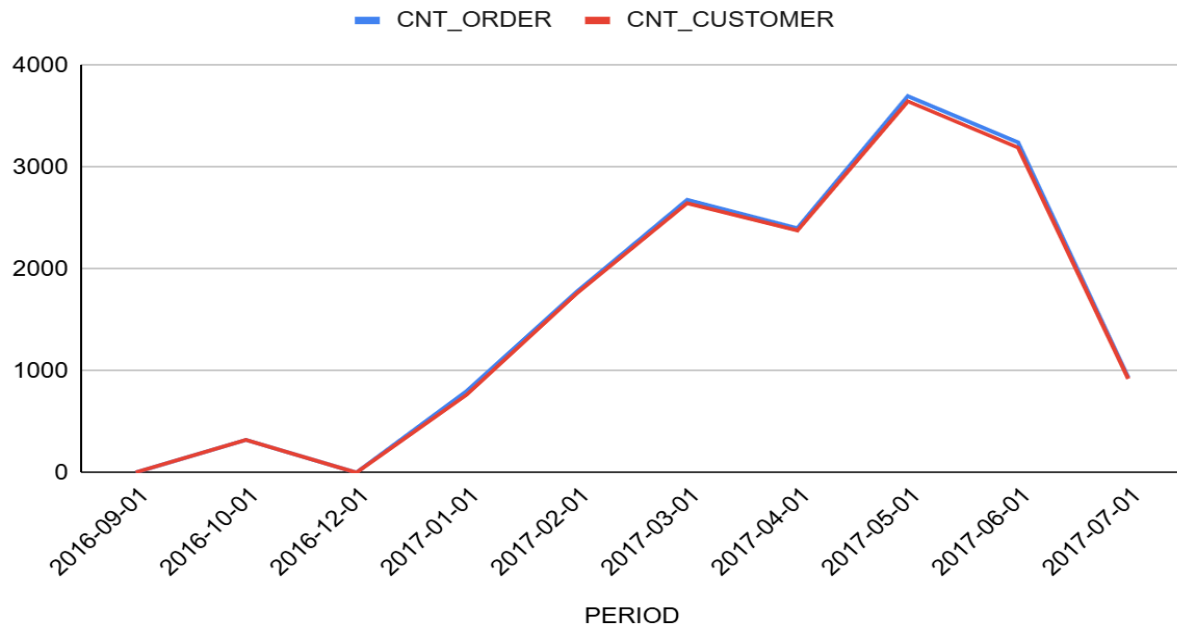
count(distinct od.order_id) as cnt_order ,
count(distinct cd.customer_unique_id) as cnt_customer ,
coalesce(sum(op.payment_value),0) as revenue ,

coalesce(sum(op.payment_value) / count(distinct case
when od.order_id is null or
od.order_id = ''
then 'a' else od.order_id end),0)
as avg_revenue_order,
coalesce(SUM(op.payment_value) / COUNT(distinct case
when cd.customer_unique_id is
null or
cd.customer_unique_id = ''
then 'a' else
cd.customer_unique_id end),0) as avg_revenue_customer
from orders_dataset as od
left join customers_dataset as cd
on od.customer_id = cd.customer_id
left join orderpayment as op
on od.order_id = op.order_id

group by 1
order by 1
```

For study different period chart we are using an Excel sheet for the chart apply formulae

=IFS(\$A\$1="YEAR", DATE(YEAR(B3),1,1), \$A\$1="MONTH", DATE(YEAR(B3), MONTH(B3), 1), \$A\$1="DAY", B3)



CONCLUSION - After analyzing the chart, we can observe that the number of orders is equal to the number of customers. This indicates that each customer has placed only one order, meaning there are no repeat customers. The absence of repeat orders is a concern, as it suggests low customer retention and may negatively impact long-term business growth.

4. Backend Analysis

1. **Deals with nulls** : NULL represents missing, unknown, or unavailable data. Handling NULL values correctly is important to ensure accurate reports, calculations, and business decisions.

Code -

```
delivery_report as
(
  select od.order_id,nullif(trim(od.order_estimated_delivery_date), ''):: date as
  estimated_date ,coalesce(nullif(trim(od.order_delivered_customer_date),
  '')::date,ld.latest_delivery::date) as final_delivery_date
```

```
from orders_dataset od
cross join latest_date ld),
```

2. Delivery_report

This section provides insights into delivery performance by comparing the estimated delivery date with the actual delivery date. It identifies delivery delays, calculates the number of delay days, analyzes the revenue generated from delayed orders, and evaluates the financial impact of delivery delays on the business.

MATRICES-

1)Estimated delivery date :

The expected date of delivery.

2)Actual delivery date :

The date the order was actually delivered.

3)Count estimated delivery dates

Total no. of estimated delivery dates.

4)Count actual delivery dates:

Total no. of actual delivery dates.

5)On-time deliveries :

Total number of orders delivered on or before the estimated delivery date.

6)Delayed deliveries :

Total number of orders delivered after the estimated delivery date.

7)Avg. Delay days :

The average number of days by which delayed orders exceeded the estimated deliveries date,

- Avg. Revenue per delayed orders :

Avg. revenue generated from orders that experienced delivery delays.

- Avg. Revenue per customer:

Average revenue generated per customer.

CODE-

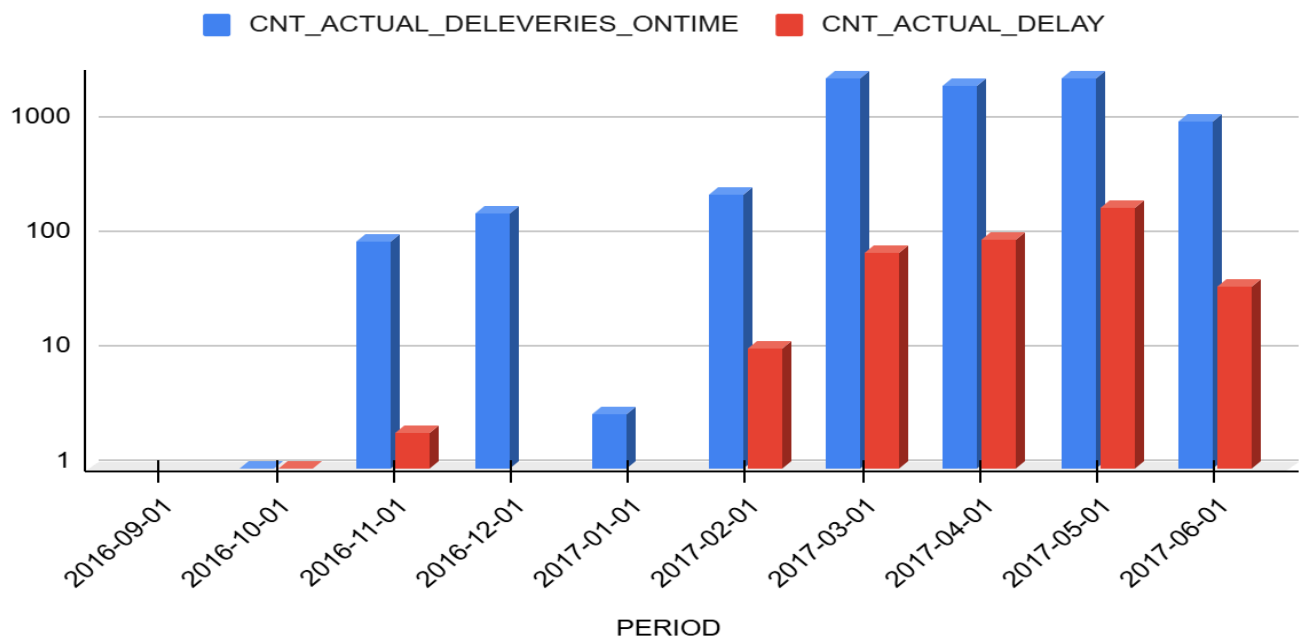
delay_reports as

```
(
  select od.order_id, od.order_status, dr.estimated_date , dr.final_delivery_date,
         case when od.order_status = 'canceled' then 0
              else dr.final_delivery_date - dr.estimated_date end as
         Delay_days
from delivery_report as dr
left join orders_dataset as od
on od.order_id = dr.order_id)
```

```

delay_output as
(
  select date_trunc('month' dr.estimated_date) as period,
         count(distinct dr.order_id) as cnt_estimated_deliveries,
         count(distinct case when dr.order_status = 'delivered' then
                               dr.order_id end) as cnt_actual_deliveries
         count(distinct case when dr.order_status = 'delivered' and
                               dr.delay_days <= 0 then dr.order_id end)
         as cnt_ontime_deliveries,
         count(distinct case when dr.delay_days > 0 then dr.order_id end)
         as cnt_delay_day,
         sum(case when dr.delay_days > 0 then dr.delay_days else 0 end)
         /nullif(count(distinct case when dr.delay_days > 0 then
                               dr.order_id end),0) as avg_delay_days,
         sum(case when dr.delay_days > 0 then f.payment_value else 0
              end)/nullif(count(distinct case when dr.delay_days > 0
              then dr.order_id end), 0) as avg_revenue_delayed_orders,
         sum(case when dr.delay_days > 0 then f.payment_value else 0
              end)/nullif(count(distinct case when dr.delay_days > 0
              Then cd.customer_unique_id end), 0)
         as avg_revenue_delayed_customers
  from delay_reports dr
       left join orderpayment f
         on f.order_id = dr.order_id
       left join orders_dataset od
         on od.order_id = dr.order_id
       left join customers_dataset cd
         on od.customer_id = cd.customer_id
  group by 1
  order by 1

```

3. All_period

Ensures no missing dates in the final dashboard.

```
all_periods as
(
  select order_period as period
  from front_end

  Union

  select period
  from delay_output
)
```

4. Final_output

```

select ap.period :: date,
       coalesce(f.cnt_order , 0) as cnt_order,
       coalesce(f.cnt_customer , 0) as cnt_customer,
       coalesce(f.revenue , 0) as order_revenue,
       coalesce(f.avg_revenue_order , 0) as avg_revenue_order,
       coalesce(f.avg_revenue_customer , 0) as avg_revenue_customer ,
       coalesce(d.cnt_estimated_deliveries, 0) as cnt_estimated_delivereies,
       coalesce(d.cnt_actual_deliveries , 0) as cnt_actual_deliveries,
       coalesce(d.cnt_ontime_deliveries , 0) as cnt_actual_deliveries_ONTIME,
       coalesce(D.cnt_delay_day , 0) as cnt_ACTUAL_delay,
       coalesce(d.avg_delay_days , 0) as avg_delay_days,
       coalesce(d.avg_revenue_delayed_orders , 0) as avg_revenue_delayed_orders,
       coalesce(d.avg_revenue_delayed_customers , 0) as
avg_revenue_delayed_customers
from all_periods as ap
  left join front_end as f
        on ap.period = f.order_period
  left join delay_output as d
        on ap.period = d.period
order by 1

```

5. DELAY REPORTS

- breakdown into delivery categories as

- Ontime
 1. Before time
- Delays
 1. 1-3 delay days
 2. 4-7 delay days
 3. Above a week delay

```

Delivery_partition as
(
select order_id , estimated_date , final_delivery_date , delay_days ,
       Case
         when delay_days < 0 then 'Before time'
         when delay_days = 0 then 'ontime'
         when delay_days between 1 and 3 then '1-3 days delay'
         when delay_days between 4 and 7 then '4-7 days delay'
         when delay_days > 7 then 'Above a week delay'
         else 'data not available'
       end as delay_period

from delay_reports
order by

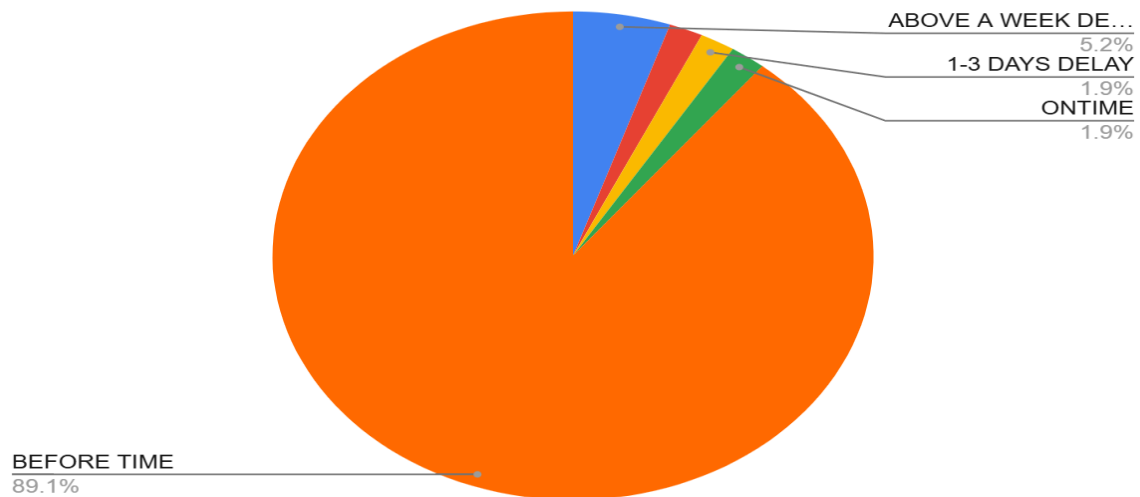
```

```

case
  when delay_days < 0 then 1
  when delay_days = 0 then 2
  when delay_days between 1 and 3 then 3
  when delay_days between 4 and 7 then 4
  when delay_days > 7 then 5
  else 6
end asc
),

```

total_days vs. delay_period



6. Sell maximum product - Find the product or products with the highest sales (maximum demand)

```

with product_sell as (
select
  pc.product_category_name_english AS product_name,
  count(*) as total_sell
from products_dataset pd
Left join product_category_name_translation pc
  on pd.product_category_name = pc.product_category_name
group by pc.product_category_name_english
)
Select
  'Maximum' AS sell_type,
  product_name,

```

```

    total_sell
from product_sell
where total_sell = (Select MAX(total_sell) from product_sell)

Union all

Select
    'Minimum' AS sell_type,
    product_name,
    total_sell
from product_sell
where total_sell = (Select MIN(total_sell) from product_sell);

```

Output-

sell_type	product_name	total_sell
Maximum	bed_bath_table	3029
Minimum	cds_dvds_musicals	1

7)STATE-WISE DISTRIBUTION OF REVENUE

```

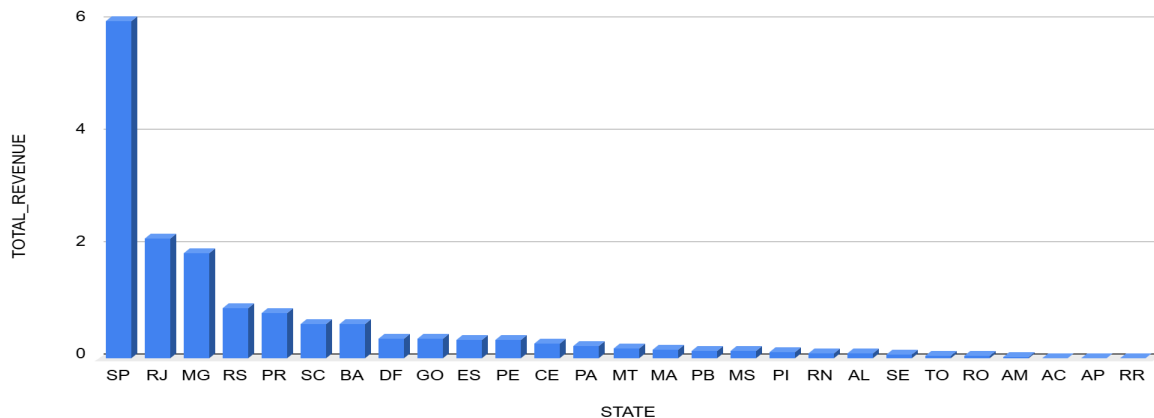
state_sales AS
(
    Select cd.customer_state ,sum(op.payment_value) AS total_revenue
    from orders_dataset as od
        Left join customers_dataset as cd
            on od.customer_id = cd.customer_id
        Left join orderpayment as op
            on od.order_id = op.order_id
    Group by 1
    Order by 2 desc
)

select * from state_sales
limit 1

```

Output-

STATE VS TOTAL_REVENUE



7. REGIONAL DELAYED REPORT

CODE -

DELAYED_REPORT AS

```
(
Select cd.customer_state , cd.customer_city,
      coalesce(count(distinct dr.order_id),0) as total_cnt,
      coalesce(sum(op.payment_value),0) as payment_city,
      coalesce(count(distinct case when dr.final_delivery_date >
        dr.estimated_date then 1 end),0) as cnt_delay,
      coalesce(count(distinct case when dr.final_delivery_date <=
        dr.estimated_date then 1 end),0) as cnt_ontime,
      coalesce(sum(case when dr.final_delivery_date > dr.estimated_date
        then op.payment_value else 0 end),0)
      as Total_revenue_delay,
      coalesce(sum(case when dr.final_delivery_date > dr.estimated_date
        then op.payment_value else 0 END) / nullif(count(distinct
        case when dr.final_delivery_date > dr.estimated_date
        then Dr.order_ID end), 0),0) as avg_revenue_delay,
      coalesce(sum(case when dr.final_delivery_date > dr.estimated_date
        then op.payment_value else 0 end) / nullif(count(distinct
        Case when dr.final_delivery_date > dr.estimated_date
        then cd.customer_unique_id end),0),0)
      As Revenue_customer_state_delay
```

from delivery_report as dr

```
left join orderpayment as op
  on dr.order_id = op.order_id
left join orders_dataset as od
  on dr.order_id = od.order_id
left join customers_dataset as CD
  on od.customer_id = cd.customer_id
group by 1,2
order by total_cnt desc
```

Analysis Workflow-

1. We analyzed customer visits on a daily, weekly, monthly, and yearly basis to understand overall customer activity and purchasing trends.
This suggests a low customer retention rate, which is a significant concern for the business.
2. Check performance by comparing estimated and actual delivery dates. Classify deliveries as on-time or delayed, and calculate the number of ontime and delay days.
Improve dispatch and logistics operations to reduce delivery delays and maintain a high on-time delivery rate.
3. Analyze regional sales performance by identifying the regions with the highest and lowest sales.
4. Measure overall revenue generated and compare revenue from on-time deliveries with revenue from delayed deliveries to understand the financial impact of delivery performance.
5. Summarize the findings and provide recommendations to improve customer retention, delivery efficiency, and revenue growth.